

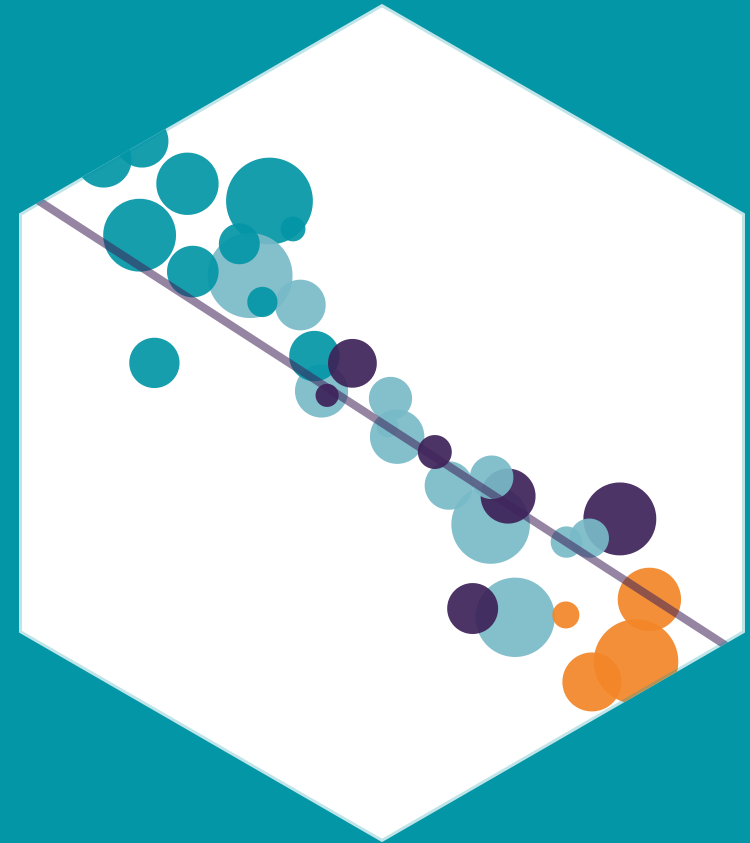
ECONOMETRICS

TA SESSION 2

Barcelona School of Economics

Lucia Sauer

✉ lucia.sauer@bse.eu



Overview

- Conditional means
- OLS in matrix algebra
- OLS using Python preset functions
- Plotting observations and fitted lines
- Verify some numerical property

Why These Topics?

🎯 Conditional Means

- Foundation of regression: OLS estimates the conditional mean of Y given X

📊 OLS in Matrix Algebra

- Build intuition for how OLS works beyond formulas and preset functions

✅ Numerical Conditions

- Check core properties of OLS to validate results and understand residual behavior

From averages to regression

Conditional Mean

Conditional Mean

Population concept: The *conditional mean* of Y given $X = x$ is the expected value of Y in the sub-population where $X = x$:

$$E[Y | X = x]$$

Sample estimate: The *sample conditional mean* is the average of all observed Y_i for which $X_i = x$:

$$\hat{E}[Y | X = x] = \frac{1}{N_x} \sum_{i: X_i = x} Y_i$$

where N_x is the number of observations with $X_i = x$.

👉 The OLS estimator aims to model the *conditional mean function*, i.e., $E[Y | X]$, as a function of X .

Example: House Prices and Size

Description dataset

```
1 import wooldridge as woo
2 import pandas as pd
3
4 df = woo.dataWoo("hprice1")
5 df.sample(8, random_state=1)
```

	price	assess	bdrms	lotsize	sqrft	colonial	lprice	lassess	llotsize	lsqrft
60	225.0	291.000000	3	7566.0	1567	0	5.416101	5.673323	8.931419	7.356918
69	220.0	263.399994	3	6300.0	1587	1	5.393628	5.573674	8.748305	7.369601
53	180.0	221.500000	3	6017.0	1812	1	5.192957	5.400423	8.702344	7.502186
54	250.0	268.399994	3	8410.0	1722	1	5.521461	5.592478	9.037177	7.451241
70	215.0	300.399994	3	11554.0	1694	0	5.370638	5.705115	9.354787	7.434848
85	202.5	258.100006	3	6341.0	1574	0	5.310740	5.553347	8.754792	7.361375
59	316.0	314.600006	4	5539.0	2343	0	5.755742	5.751302	8.619569	7.759187

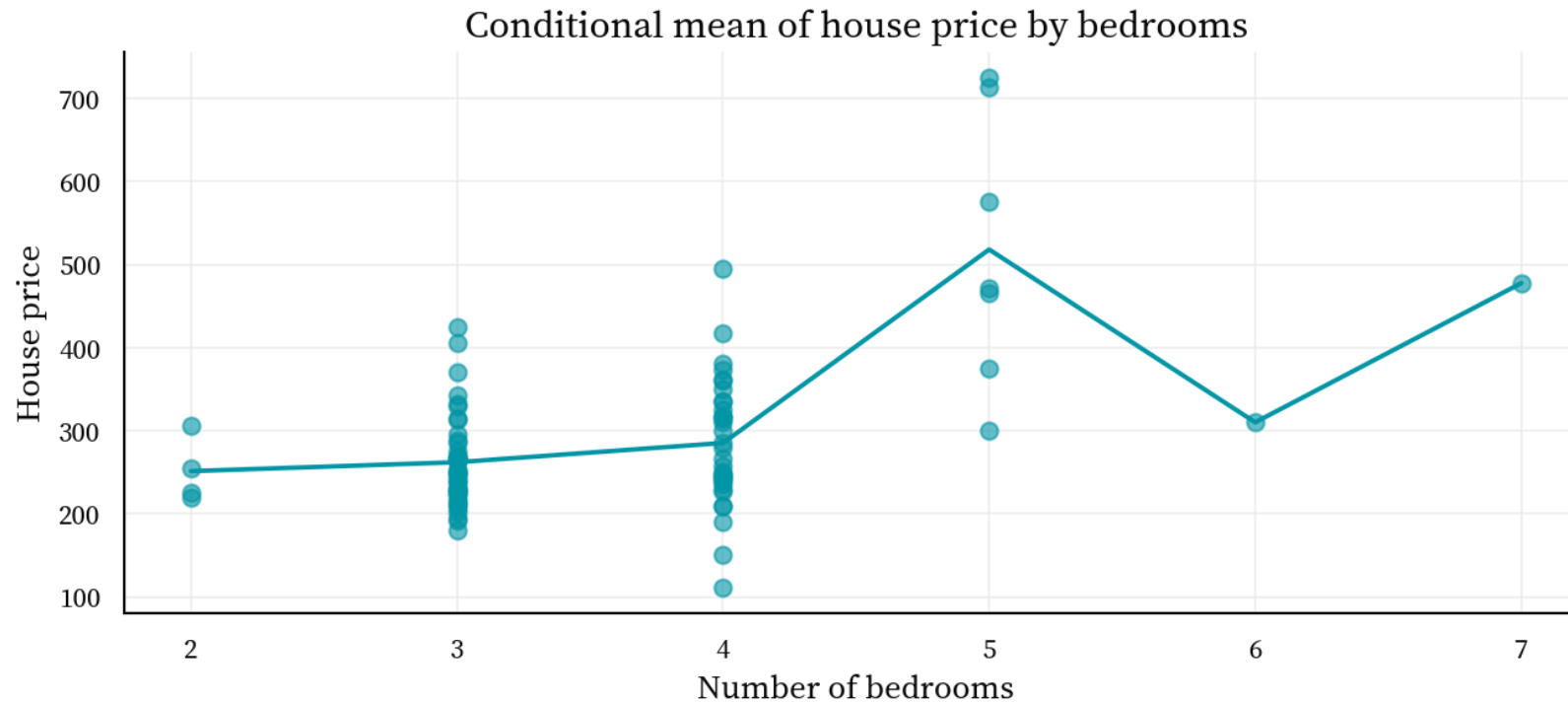
Compute Conditional Mean in Python

```
1 df_grouped = (  
2     df.groupby("bdrms", as_index=False)["price"]  
3     .mean()  
4     .rename(columns={"price": "mean_price"})  
5 )  
6 df_grouped
```

	bdrms	mean_price
0	2	251.250000
1	3	261.979167
2	4	285.163667
3	5	518.003571
4	6	310.000000
5	7	477.500000

Plot Conditional Mean in Python

```
1 import matplotlib.pyplot as plt
2
3 fig, ax = plt.subplots(figsize=(9, 4.5))
4 ax.scatter(df["bdrms"], df["price"], color=bse_style.TEAL, alpha=0.6, s=45)
5 ax.plot(df_grouped["bdrms"], df_grouped["mean_price"], color=bse_style.TEAL, linewidth=1.8)
6 ax.set_title(r"$\hat{E}[price \mid bdrms]$")
7 ax.set_xlabel("Number of bedrooms")
8 ax.set_ylabel("Average house price")
9 plt.show()
```



From averages to a fitted model

OLS in Matrix Algebra

Using our dataset, we can write the model for a representative observation i :

$$price_i = \beta_1 + \beta_2 \cdot bdrms_i + \beta_3 \cdot sqrf t_i + \beta_4 \cdot colonial_i + \varepsilon_i$$

We can express this model for a set of n representative observations in matrix form as:

$$y = X\beta + \varepsilon$$

where:

$$\begin{bmatrix} price_1 \\ price_2 \\ \vdots \\ price_n \end{bmatrix}, \begin{bmatrix} 1 & bdrms_1 & sqrf t_1 & col_1 \\ 1 & bdrms_2 & sqrf t_2 & col_2 \\ \vdots & \vdots & \vdots & \\ 1 & bdrms_n & sqrf t_n & col_n \end{bmatrix}, \begin{bmatrix} \beta_1 \\ \beta_2 \\ \beta_3 \\ \beta_4 \end{bmatrix}, \begin{bmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \vdots \\ \varepsilon_n \end{bmatrix}$$

Exercise: OLS in Matrix Algebra

Exercises

1. Estimate the OLS coefficients using matrix algebra.
2. Compute the fitted values and OLS residuals.
3. Calculate the Sum of Squared Errors (SSE).
4. Compute the R^2 statistic.

1. Estimate the OLS Coefficients

Starting from the OLS objective function:

$$\min_{\beta^{\sim}} SSE(\beta^{\sim}) = \min_{\beta^{\sim}} \sum_{i=1}^n (y_i - x_i' \beta^{\sim})^2 = \min_{\beta^{\sim}} (y - X\beta^{\sim})' (y - X\beta^{\sim})$$

Taking FOC and solving gives $\hat{\beta}$:

$$\hat{\beta} = \left(\sum_{i=1}^n x_i x_i' \right)^{-1} \sum_{i=1}^n x_i y_i$$

Or equivalently, in matrix form:

$$\hat{\beta} = (X'X)^{-1} X'y$$

Compute OLS coefficients in Python

```
1 import numpy as np
2
3 df["intercept"] = 1
4 X = df[["intercept", "bdrms", "sqrft", "colonial"]].to_numpy()
5 y = df["price"].to_numpy()
6 n = X.shape[0]
7
8 # matrix form: X'X and X'y
9 X_tX = X.T @ X
10 X_ty = X.T @ y
11 beta_hat = np.linalg.solve(X_tX, X_ty)
12
13 print(np.round(beta_hat, 2))
```

```
[-21.55  12.49   0.13  13.08]
```

Fitted model:

$$\hat{price}_i = -21.55 + 12.49 \cdot bdrms_i + 0.13 \cdot sqrft_i + 13.08 \cdot colonial_i$$

where the dependent variable `price` is in **\$1000s**.

2. Compute the fitted values and OLS residuals

$$\hat{y} = X\hat{\beta}$$

```
1 # Fitted values
2 y_hat = X @ beta_hat
```

$$\hat{\varepsilon} = y - \hat{y}$$

```
1 # Residuals
2 epsilon_hat = y - y_hat
```

Units

1. In what units are the fitted values $\hat{y}$?
2. In what units are the residuals $\hat{\varepsilon}$?

3. Calculate the Sum of Squared Errors (SSE)

$$SSE = \varepsilon^{\wedge} \varepsilon^{\wedge}$$

Note that this is exactly the same as:

$$SSE = \sum_{i=1}^n (\varepsilon_i^{\wedge})^2 = \sum_{i=1}^n (y_i - y_i^{\wedge})^2$$

```
1 # Sum of Squared Errors
2 SSE = epsilon_hat.T @ epsilon_hat
3 round(SSE, 2)

np.float64(334982.95)
```

SSE Units

Note: The SSE is in the squared units of the dependent variable (here, the price in 1000s of dollars).

4. Compute the R^2 statistic

$$R^2 = 1 - \frac{SSE}{SST}$$

where

$$SST = (y - \bar{y}1)'(y - \bar{y}1)$$

```
1 # Total Sum of Squares
2 y_bar = y.mean()
3 SST = (y - y_bar).T @ (y - y_bar)
4 r2 = 1 - SSE / SST
5 round(r2, 4)

np.float64(0.635)
```

R^2 interpretation

Note: The R^2 is unit free. It tells us that about 64% of the sample variation in house price is explained by the model.

Same model, shorter code

3. Python Preset Functions

Preset Functions

All the operations we did in matrix algebra can be done using preset functions in Python.

- Python: `statsmodels` library, specifically the `OLS` class from `statsmodels.api`.

Code example in Python

```
1 import statsmodels.api as sm
2 from statsmodels.formula.api import ols
3
4 # Estimate the model with the ols formula API
5 model = ols("price ~ bdrms + sqrft + colonial", data=df).fit()
6 beta_hat = model.params
7 y_hat = model.fittedvalues
8 epsilon_hat = model.resid
9
10 sse = (epsilon_hat**2).sum()
11 r2 = model.rsquared
12 print("Coefficients (betas):\n", beta_hat)
13
14 print("\nSSE:", round(sse, 2))
15 print("R^2:", round(r2, 4))
```

Coefficients (betas):

Intercept	-21.552413
bdrms	12.487493
sqrft	0.129849
colonial	13.077547

dtype: float64

SSE: 334982.95

R^2: 0.635

4. Plotting Observations and Fitted Line

For a simple model of $K = 2$, estimate the model and plot the observations and the fitted line.

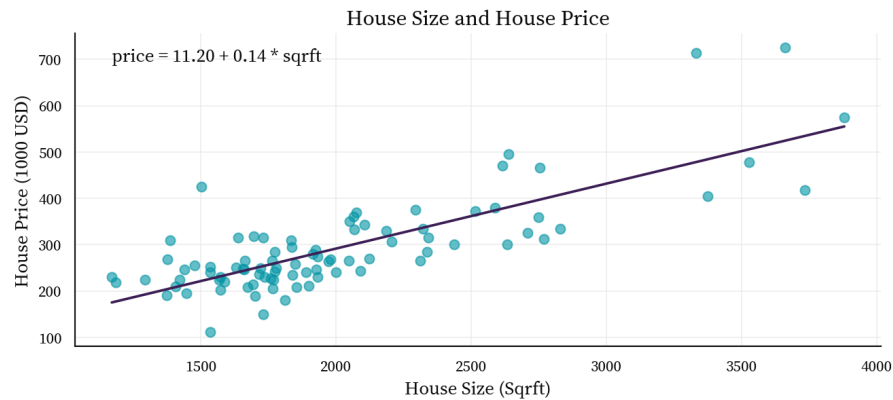
$$price_i = \beta_1 + \beta_2 \cdot sqft_i + \epsilon_i$$

```
1 # estimate the model
2 # first we need to estimate the simple model
3 model = ols("price ~ sqft", data=df).fit()
4 y_hat = model.fittedvalues
5 beta_hat = model.params
6 beta_hat
```

```
Intercept    11.204145
sqft          0.140211
dtype: float64
```

Fitted line for house size and price

```
1 # plot observations with scatter and fitted line
2 fig, ax = plt.subplots(figsize=(9, 4.5))
3
4 # scatter with raw data
5 ax.scatter(df["sqrft"], df["price"], color=bse_style.TEAL, s=40, alpha=0.65)
6
7 # fitted line
8 x_vals = np.array(ax.get_xlim())
9 ax.plot(x_vals, beta_hat["Intercept"] + beta_hat["sqrft"] * x_vals,
10         color=bse_style.PURPLE, linewidth=1.8)
11
12 # labels and styling
13 ax.set_title("House size and price")
14 ax.set_xlabel("sqrft")
15 ax.set_ylabel("house price in 1000 usd")
16 plt.show()
```



Reading the residuals

5. Numerical Properties of OLS

Numerical Property of OLS

These properties are independent of the statistical assumptions, they are purely mathematical properties of the OLS estimator, that hold given a sample.

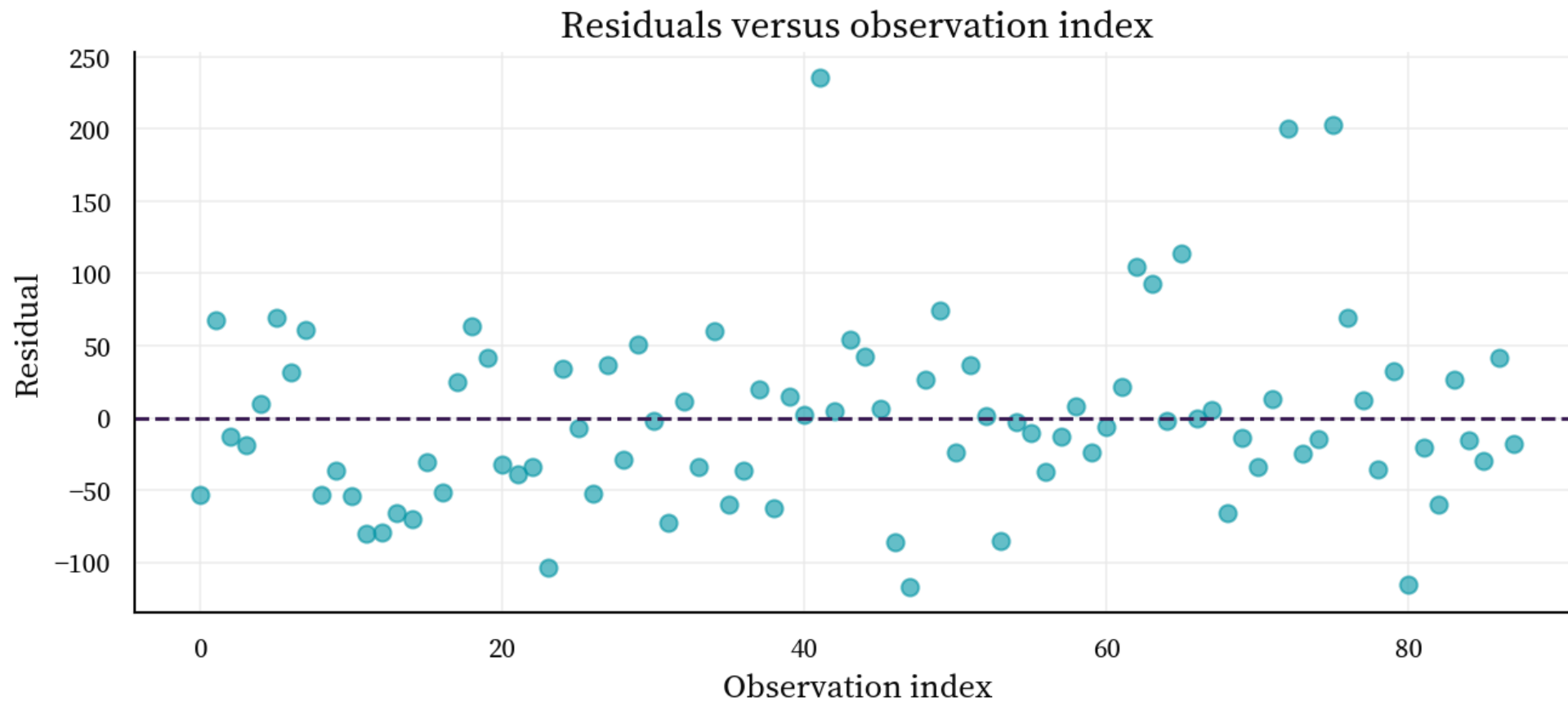
1.
$$\sum_{i=1}^n \hat{\varepsilon}_i = 0$$

2. OLS is unit-dependent, hence SSE is also unit-dependent.

1. Sum of residuals is zero

```
1 epsilon_hat = model.resid
2 round(epsilon_hat.sum(), 2)
np.float64(-0.0)
```

Illustration:



2. OLS is unit-dependent, hence SSE is unit-dependent

Our dependent variable is expressed in \$1000s, so let's scale it to actual dollars and see how the SSE changes.

Coefficients in thousands of dollars: [11.2041, 0.1402]

Coefficients in dollars: [11204.145, 140.211]

SSE in thousands of dollars: 334,982.95

SSE in dollars: 348,053,431,609.34